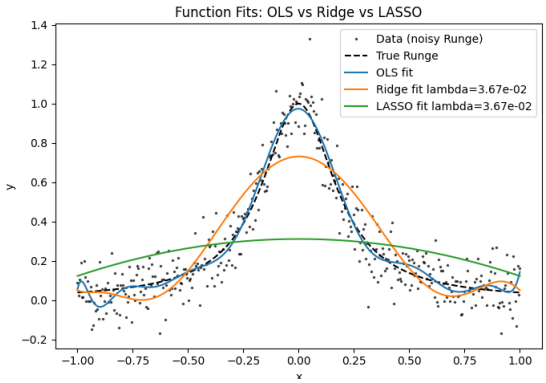
UiO : University of Oslo

A Comparative Study of Regression and Classification Using a Self-Implemented Feed-Forward Neural Network

Project 2 on Machine Learning (Deadline: November 10 (midnight), 2025)

Author:
Theodor ST#: 0501579 &
Bartosz ST#: 703557 & Samson
ST#: 555652

Supervisor: *Prof. Morten Hjorth-Jensen*



October 31, 2025

Contents

List of Figures

Abstract

In this project, we develop a feed-forward neural network (FFNN) from scratch to study both regression and classification tasks. Using the one-dimensional Runge function, we compare our FFNN against Ordinary Least Squares (OLS), Ridge, and Lasso regression to evaluate how non-linear architectures compare to classical methods. For classification, we train the same network on the MNIST dataset using a Softmax output layer. The FFNN achieves an average test MSE of 0.005 on the Runge function—about `25%` lower than the best regularized linear model—and an accuracy of `96.9%` on MNIST, matching a comparable scikit-learn MLP implementation. Our analysis demonstrates how activation choice (ReLU vs. Sigmoid) and optimization strategy (Adam vs. stochastic gradient descent) affect convergence speed and generalization.

NOTE: This is just a random exemplary data to showcase the structure of the abstract.

1 Introduction

Machine learning methods are widely used to model complex relationships between variables and to extract patterns from large datasets. Neural networks form the foundation of many modern machine-learning algorithms, as they are capable of approximating highly nonlinear functions. In this project, we develop a feed-forward neural network (FFNN) from scratch to study both regression and classification tasks. Implementing the model manually allows us to better understand how the underlying mathematical components—gradient descent, activation functions, and backpropagation—combine to form a flexible learning framework.

A central motivation for this study is to compare our self-developed FFNN with the classical regression methods analyzed in Project 1. While Ordinary Least Squares (OLS) and its regularized extensions (Ridge and Lasso) capture linear dependencies effectively, they struggle to approximate nonlinear functions such as the Runge function. Neural networks can represent such relationships through a composition of weighted transformations and nonlinear activations, which makes them well-suited for modeling complex mappings. By training the FFNN on the Runge benchmark, we aim to evaluate how its flexibility translates into prediction accuracy and generalization. [Place for quantitative results comparing the FFNN's MSE to the OLS/Ridge/Lasso baselines.]

In addition to regression, the project extends to a classification task using the MNIST dataset of handwritten digits. This allows us to investigate how the same network architecture can be applied to both continuous and categorical outputs. For regression, we evaluate model performance using the mean squared error (MSE), while for classification, we use accuracy and confusion matrices. Various activation functions (Sigmoid, ReLU, and Leaky ReLU) and optimization methods (Gradient Descent, RMSProp, and Adam) are tested to analyze their effects on convergence and performance. [Place for results describing classification accuracy and the effect of different activation functions and optimizers.]

We further explore the influence of network architecture—number of layers and nodes per layer—and include L_1 and L_2 regularization terms to study how these affect overfitting and generalization. Finally, our implementation is benchmarked against scikit-learn and PyTorch to verify correctness and performance. [Place for a brief comparison with library-based results and observations about gradient verification or runtime differences.]

The main objectives of the project were:

- Analytical warm-up: Deriving and implementing cost functions and their gradients for MSE and cross-entropy losses, and reviewing the gradient descent methods from Project 1.
- Neural network implementation: Writing an FFNN from scratch with a flexible number of layers and nodes, supporting Sigmoid, ReLU, and Leaky ReLU activations.
- Comparison with existing software: Testing our implementation against equivalent models in scikit-learn and PyTorch, and verifying gradients using autograd.
- Activation and depth studies: Investigating how network depth and activation choice affect learning outcomes and overfitting behavior.
- Regularization analysis: Introducing L_1 and L_2 penalty terms and comparing results with Ridge and Lasso regressions from Project 1.
- Classification analysis: Adapting the FFNN to perform multiclass classification on the MNIST dataset using Softmax and cross-entropy loss.

- Critical evaluation: Assessing the advantages and limitations of neural networks compared with linear and logistic models, and evaluating how optimization algorithms influence training stability and generalization.

The remainder of this report is organized as follows. Section II describes the theoretical background and the mathematical formulation of the neural network. Section III presents the implementation details and experimental setup. Section IV discusses the results for regression and classification, while Section V concludes with a summary of findings and future work.

2 Methods

This section describes the theoretical framework and mathematical formulation of the models used in the project. [Place for a short introductory paragraph once the section is written in full.]

2.1 Feed-Forward Neural Networks

Artificial neural networks consist of layers of nodes that transform inputs through weighted connections and nonlinear activation functions. In this project, we focus on a fully connected feed-forward architecture trained by minimizing a cost function with respect to its parameters using gradient-based optimization. [Place for formal description of layer equations, matrix notation, and the general forward-propagation formula.]

2.2 Cost Functions

For regression problems, the network minimizes the mean squared error (MSE) between predictions and target values. For classification, we use the cross-entropy loss combined with the Softmax output function. [Place for mathematical expressions and short explanation of their properties (convexity, gradient).]
Regularization terms based on the L_1 and L_2 norms are included to penalize large weights and control overfitting. [Place for definition of the regularized loss functions and gradients.]

2.3 Activation Functions

Nonlinear activation functions allow neural networks to model complex, nonlinear relationships. We test the Sigmoid, ReLU, and Leaky ReLU functions, each with different gradient behavior and suitability for deep networks. [Place for expressions and first derivatives.]

2.4 Optimization Algorithms

The model parameters are optimized using variants of gradient descent. We implement both stochastic and adaptive approaches, including RMSProp and Adam, which adjust learning rates during training. [Place for update-rule equations and notes on convergence or computational cost.]

2.5 Regularization and Generalization

Regularization improves generalization by constraining model complexity. We study both L_1 (sparsity-inducing) and L_2 (weight-decay) penalties, comparing their effects on regression and classification accuracy. [Place for later discussion of overfitting trends and bias–variance connection.]

2.6 Data Preprocessing and Scaling

Before training, all input features are scaled to zero mean and unit variance to ensure stable optimization. For regression, [what we do for regression]; for classification, [what we do for classification]. [Place for more details on dataset size, splits, etc.]

2.7 Evaluation Metrics

For regression, we use the MSE and R2 score; for classification, we report accuracy and analyze the confusion matrix. [Place for definitions of metrics and a short comment on their interpretation.]

2.8 Model Comparison and Validation

The performance of our own implementation is compared with equivalent models from scikit-learn and PyTorch to ensure correctness and numerical stability. We also verify the analytical gradients by comparing them with autograd. [Place for quantitative comparison]

3 Implementation

3.1 Code

See the appendix A for all the source code used in this report. All the code lives in a single .ipynb file on GitHub. The code is yet to be provided.

3.2 Use of AI tools

ChatGPT has been used to generate a text skeleton to expedite the writing process and also as a debugging tool to aid in writing code and resolving some coding issues. See the appendix A for the AI chat-log.

4 Results and Discussion

Here is the place for future results. So far we present the required figures along with answers to posed questions.

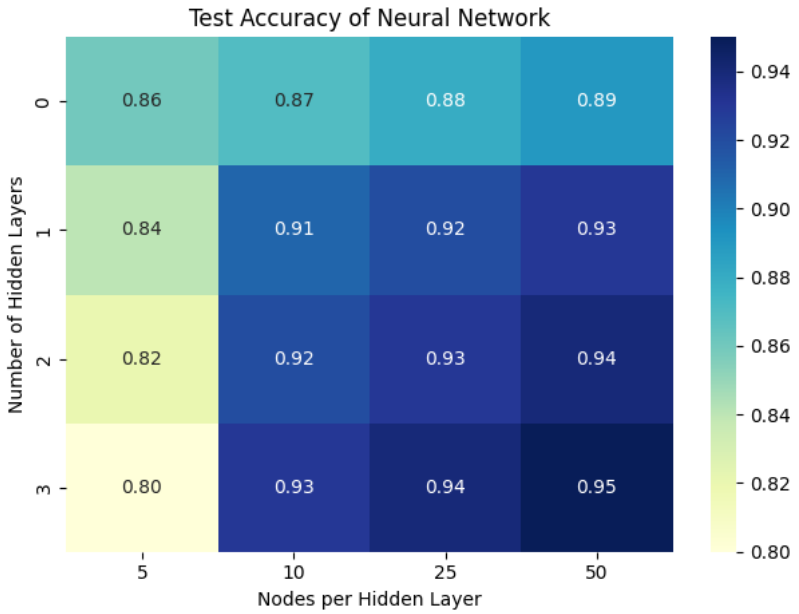


Figure 1: Heatmap showing the test accuracy of a neural network with [0, 1, 2, 3] hidden layers and [5, 10, 25, 50] nodes per hidden layer.

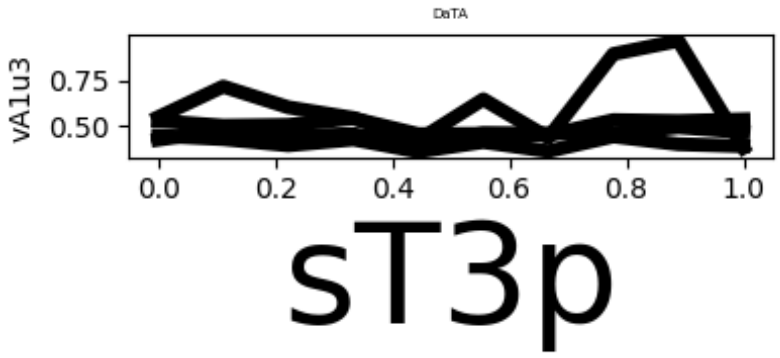


Figure 2: Figure 1

Issues with 2:

- Vague and inconsistent title: "DaTA" is meaningless, oddly capitalized, and provides no context about what is plotted.
- Inconsistent and unclear labels: Axis labels ("sT3p" and "vA1u3") are nonsensical and inconsistently formatted.
- No units or defined quantities: Neither axis specifies what is being measured, leaving the figure uninterpretable.

- Unreadable text: The title font is overly large while the axis labels are too small, breaking visual balance.
- Poor color choice: All lines use the same black color, making them visually identical and impossible to distinguish.
- Missing legend: The reader has no way to identify which line represents which data.
- Overlapping data: Multiple lines intersect frequently, further reducing clarity and making trends indistinguishable.
- Uninformative scale and ticks: The axes have default scales and tick marks that convey no meaningful information.
- Cramped layout: The small figure size (4×2 inches) makes the plot crowded and hard to read.
- Low resolution: Saved at 100 dpi, resulting in blurry or pixelated rendering when viewed or printed.
- Confusing caption: The label "Figure 2: Figure 1" is inconsistent, uninformative, and misleading.

---

5 Conclusion

The conclusions are yet to be conclusively concluded :)

A Source Code

The full source code for this projectwill be available on GitHub:

https://github.com/Twaal/applied-data-analysis-and-ml/blob/main/exercises/E10/Data-Analysis-and-Machine-Learning-FYS-STK4155-Project-2.pdf

References